🧪 GUÍA DE LABORATORIO DE SIMULACIÓN CAPITULO 5 - Corke

Navegación de Robots Móviles: De lo Reactivo a ROS2 Nav2

🎯 1. Objetivos

Al finalizar este laboratorio, el estudiante será capaz de:

- Implementar estrategias de navegación reactiva
- Analizar algoritmos de evasión de obstáculos (Bug2)
- Implementar planificación de trayectorias con A*
- Aplicar planificación en mapas de ocupación
- Integrar navegación autónoma en ROS2 (Nav2)

🧠 2. Contexto

La navegación robótica consiste en guiar un robot desde una posición inicial hasta un objetivo, utilizando diferentes enfoques:

- **Reactivo**: basado en sensores, sin mapa
- **Basado en mapa**: con planificación óptima

En este laboratorio se explorarán ambos enfoques y su implementación en simulación y ROS2.

⚙️ 3. Requisitos

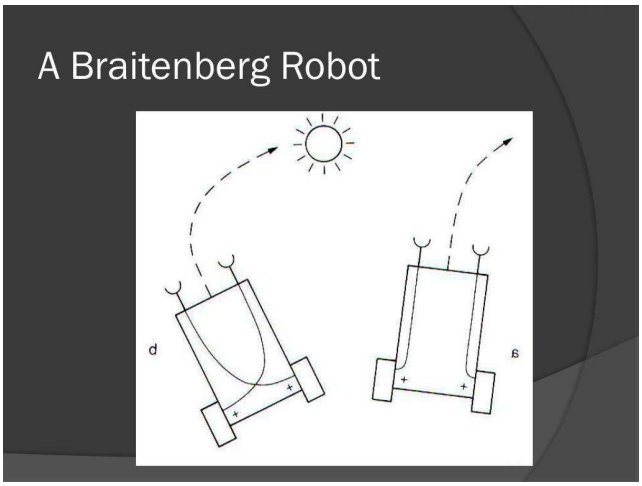
- Python 3.x
- Librerías: numpy, matplotlib, scipy
- ROS2 (Humble o equivalente)
- Paquetes:
    - nav2_bringup
    - turtlebot3

🔹 PARTE A — NAVEGACIÓN REACTIVA

🎯 Objetivo

Implementar un robot que se dirija hacia un objetivo utilizando únicamente sensores.

🧩 Actividad A1 – Implementación



Instrucciones

1. Definir un campo escalar que represente el objetivo
2. Implementar dos sensores (izquierdo y derecho) mediante simulación.
3. Diseñar una ley de control para:
    - velocidad lineal
    - velocidad angular
4. Simular la trayectoria del robot
5. Visualizar la trayectoria

Código:

```
#  Braitenberg Vehicle (completo)
import numpy as np
import matplotlib.pyplot as plt

# Campo escalar (objetivo)
def sensorfield(x, y):
   xc, yc = (60, 90)
   return 1000 / ((x-xc)**2 + (y-yc)**2 + 50)

# Parámetros
dt = 0.1
Ks = 17
Kv = 14

# Estado inicial
```

```
x, y, theta = 10, 10, 0

traj_x, traj_y = [], []

for i in range(222):
    # Sensores (offset lateral)
    d = 2
    s_left = sensorfield(x - d*np.sin(theta), y + d*np.cos(theta))
    s_right = sensorfield(x + d*np.sin(theta), y - d*np.cos(theta))

    print(s_left, s_right)

    # Control
    v = - Kv * (s_left + s_right)
    omega = Ks * (s_right - s_left)

    print(v, omega)

    # Cinemática
    x += v * np.cos(theta) * dt
    y += v * np.sin(theta) * dt
    theta += omega * dt

    traj_x.append(x)
    traj_y.append(y)
    #print(x,y,theta)

# Visualización
plt.plot(traj_x, traj_y)
plt.scatter([60], [90], c='red')
plt.title("Navigación Braitenberg")
plt.axis('equal')
plt.show()
```

❓ **Preguntas de análisis**

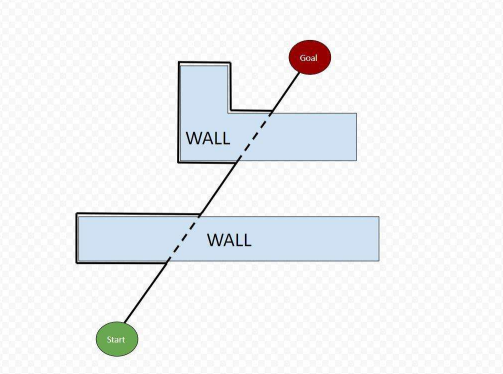
1. ¿Cómo influye la diferencia entre sensores en la dirección del robot?
2. ¿Qué ocurre cuando el robot se aproxima al objetivo?
3. ¿Qué limitaciones presenta este enfoque? (que pasa si se aumenta la cantidad de iteraciones?)

🔹 **PARTE B — NAVEGACIÓN CON BUG2**

🎯 **Objetivo**

Implementar un algoritmo de navegación con evasión de obstáculos **sin mapa global.**

🧩 **Actividad B1 – Simulación**



Instrucciones

1. Crear un mapa tipo grid con obstáculos
2. Definir punto inicial y objetivo
3. Implementar:
    - movimiento hacia el objetivo
    - detección de obstáculo
    - seguimiento del borde
    - retorno a la línea objetivo
4. Registrar la trayectoria
5. Visualizar resultados

Código:

```
#  Bug2 (simplificado en grid)
import numpy as np
import matplotlib.pyplot as plt
```

```
grid = np.zeros((20, 20))
grid[8:12, 5:15] = 1  # obstáculo

start = np.array([2, 2])
goal = np.array([18, 18])

pos = start.copy()
path = [pos.copy()]

def is_free(p):
    return grid[p[1], p[0]] == 0

for _ in range(500):
    direction = goal - pos
    # step = np.sign(direction)   #con diagonales

    if abs(direction[0]) > abs(direction[1]):
        step = np.array([np.sign(direction[0]), 0])
    else:
        step = np.array([0, np.sign(direction[1])])

    next_pos = pos + step       # ir en línea recta al objetivo

    print("pos:", pos, "next:", next_pos, "free:", is_free(next_pos))

    if is_free(next_pos):
        pos = next_pos
    else:
        # seguir borde (simplificado)   for move in [→, ↑, ←, ↓]
        for move in [np.array([1,0]), np.array([0,1]), np.array([-1,0]), np.array([0,-1])]:
            if is_free(pos + move):
                pos = pos + move
                break

    path.append(pos.copy())

    if np.linalg.norm(pos - goal) < 1:
        break

path = np.array(path)

# plt.imshow(grid, origin='lower', cmap='gray_r')

# plt.plot(path[:,0], path[:,1], 'g')

# plt.scatter(start[0], start[1], c='blue')
# plt.scatter(goal[0], goal[1], c='red')
```

```
# plt.gca().set_aspect('equal')

# plt.grid(True)
# plt.title("Bug2 - Visualización Correcta")
# plt.show()

plt.imshow(grid, origin='lower')
plt.plot(path[:,0], path[:,1], 'g')
plt.scatter(start[0], start[1], c='blue')
plt.scatter(goal[0], goal[1], c='red')
plt.title("Bug2 Simplificado")
plt.show()
```

❓ **Preguntas de análisis**

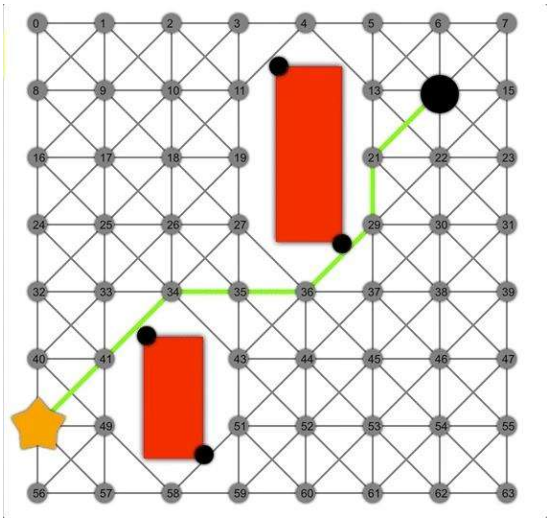
1. ¿El camino obtenido es óptimo? Justifique
2. ¿Qué sucede en entornos complejos?
3. Compare con el enfoque reactivo de la Parte A

🔹 **PARTE C — PLANIFICACIÓN CON A***

🎯 **Objetivo**

Implementar planificación óptima usando grafos.

🧩 **Actividad C1 – Implementación**



Instrucciones

1. Representar el entorno como grafo o grid
2. Implementar algoritmo A*
3. Definir función heurística
4. Calcular la ruta óptima
5. Visualizar la trayectoria

Código:

```
#  A* desde cero
import heapq
import numpy as np

grid = np.zeros((20, 20))
grid[8:12, 5:15] = 1

start = (2,2)
goal = (18,18)

def heuristic(a, b):
   return np.linalg.norm(np.array(a)-np.array(b))

def astar(grid, start, goal):
   open_list = []
   heapq.heappush(open_list, (0, start))

   came_from = {}
   cost = {start: 0}

   while open_list:
      _, current = heapq.heappop(open_list)

      if current == goal:
         break

      for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
         neighbor = (current[0]+dx, current[1]+dy)

         if (0 <= neighbor[0] < grid.shape[0] and
            0 <= neighbor[1] < grid.shape[1] and
            grid[neighbor[1], neighbor[0]] == 0):

            new_cost = cost[current] + 1
```

```
            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                priority = new_cost + heuristic(neighbor, goal)
                heapq.heappush(open_list, (priority, neighbor))
                came_from[neighbor] = current

    # reconstruir camino
    path = []
    node = goal
    while node != start:
        path.append(node)
        node = came_from[node]
    path.append(start)
    path.reverse()

    return path

path = astar(grid, start, goal)

path = np.array(path)
import matplotlib.pyplot as plt
plt.imshow(grid, origin='lower')
plt.plot(path[:,0], path[:,1], 'r')
plt.title("A* Path")
plt.show()
```

❓ **Preguntas de análisis**

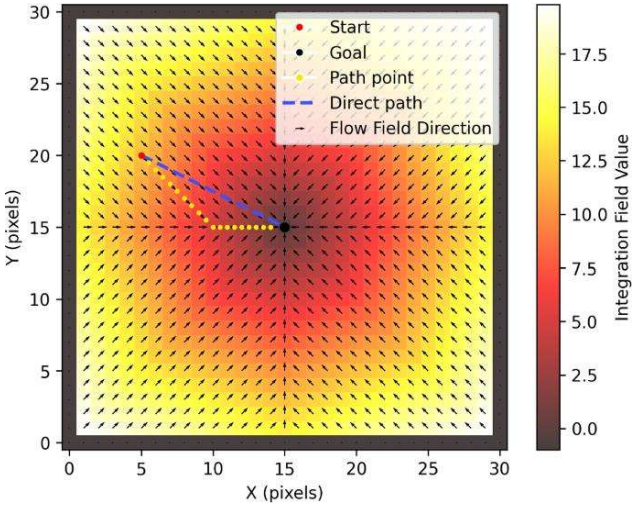
1. ¿Qué diferencia hay entre BFS, UCS y A*?
2. ¿Cuál es el papel de la heurística?
3. ¿Qué ocurre si la heurística no es adecuada?

🔹 **PARTE D — MAPAS DE OCUPACIÓN**

🎯 **Objetivo**

Generar trayectorias óptimas mediante distance transform.

🧩 **Actividad D1 – Implementación**



Instrucciones

1. Crear un mapa de ocupación
2. Definir una celda objetivo
3. Calcular el mapa de distancias
4. Obtener una trayectoria desde un punto inicial
5. Visualizar:
    - mapa de distancias
    - trayectoria

Código:

```
#  Distance Transform
import numpy as np
import matplotlib.pyplot as plt

# ==========================
# MAPA
# ==========================
grid = np.zeros((20,20))
grid[8:12, 5:15] = 1  # obstáculo

goal = (2,2)

# ==========================
# DISTANCE TRANSFORM (NO ÓPTIMO)
# ==========================
dist = np.ones_like(grid) * np.inf
```

```
dist[goal[1], goal[0]] = 0

# propagación iterativa
for _ in range(100):
    for x in range(20):
        for y in range(20):
            if grid[y,x] == 0:

                neighbors = []
                for dx,dy in [(-1,0),(1,0),(0,-1),(0,1)]:
                    nx, ny = x+dx, y+dy
                    if 0 <= nx < 20 and 0 <= ny < 20:
                        neighbors.append(dist[ny,nx])

                dist[y,x] = min(dist[y,x], min(neighbors)+1)

# =========================
# SEGUIMIENTO DE CAMINO
# =========================
start = (18,18)
pos = start
path = [pos]

for _ in range(200):
    x,y = pos

    neighbors = []
    for dx,dy in [(-1,0),(1,0),(0,-1),(0,1)]:
        nx, ny = x+dx, y+dy
        if 0 <= nx < 20 and 0 <= ny < 20:
            neighbors.append(((nx,ny), dist[ny,nx]))

    pos = min(neighbors, key=lambda x: x[1])[0]
    path.append(pos)

    if pos == goal:
        break

path = np.array(path)

# =========================
# VISUALIZACIÓN (MAPA DE CALOR)
# =========================
plt.figure(figsize=(6,6))

# heatmap de distancias
plt.imshow(dist, origin='lower', cmap='hot')
```

```
# obstáculos (en negro)
obs_y, obs_x = np.where(grid == 1)
plt.scatter(obs_x, obs_y, c='black', s=20)

# trayectoria
plt.plot(path[:,0], path[:,1], 'cyan', linewidth=2, label="trayectoria")

# puntos inicio/fin
plt.scatter(start[0], start[1], c='blue', s=80, label="inicio")
plt.scatter(goal[0], goal[1], c='green', s=80, label="objetivo")

# detalles visuales
plt.colorbar(label="Distancia al objetivo")
plt.title("Distance Transform (Iterativo) - Heatmap")
plt.legend()
plt.grid(True)
plt.gca().set_aspect('equal')

plt.show()
```

❓ **Preguntas de análisis**

1. ¿Por qué este método garantiza optimalidad?
2. ¿Cuál es su costo computacional?
3. Compare con el algoritmo Bug2

🔹 **PARTE E — IMPLEMENTACIÓN EN ROS2**

🎯 **Objetivo**

Ejecutar navegación autónoma en un robot simulado usando ROS2.

🧩 **Actividad E1 – Navegación con Nav2**

Instrucciones

1. Lanzar simulación de robot móvil
2. Inicializar sistema de navegación
3. Establecer un objetivo en RViz
4. Observar:
    - trayectoria generada
    - comportamiento del robot

🧩 **Actividad E2 – Nodo reactivo**

Instrucciones

1. Crear un nodo ROS2 en Python
2. Suscribirse al tópico /scan
3. Publicar en /cmd_vel
4. Implementar lógica de evasión de obstáculos
5. Probar en simulación

Código:

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import TwistStamped
import numpy as np

class ReactiveNavigator(Node):

    def __init__(self):
        super().__init__('reactive_navigator')

        # Suscriptor LiDAR
        self.sub = self.create_subscription(
            LaserScan,
            '/scan',
            self.scan_callback,
            10
        )

        # Publisher TwistStamped
        self.pub = self.create_publisher(
            TwistStamped,
            '/cmd_vel',
            10
        )

    def scan_callback(self, msg):
        ranges = np.array(msg.ranges)

        # Limpiar valores inválidos (inf, nan)
        ranges = ranges[np.isfinite(ranges)]
        if len(ranges) == 0:
            return
```

```
        # Sector frontal
        n = len(msg.ranges)
        front_sector = msg.ranges[n//3:2*n//3]
        front = np.min(front_sector)

        cmd = TwistStamped()

        # Header (IMPORTANTE)
        cmd.header.stamp = self.get_clock().now().to_msg()
        cmd.header.frame_id = 'base_link'  # o '' si no es requerido

        # Lógica reactiva
        if front < 0.5:
            cmd.twist.linear.x = 0.0
            cmd.twist.angular.z = 0.8
        else:
            cmd.twist.linear.x = 0.2
            cmd.twist.angular.z = 0.0

        self.pub.publish(cmd)

def main():
    rclpy.init()
    node = ReactiveNavigator()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

❓ **Preguntas de análisis**

1. ¿Qué diferencia hay entre navegación reactiva y Nav2?
2. ¿Qué función cumple el planner dentro de Nav2?
3. ¿Qué ventajas ofrece ROS2 en robótica móvil?